

LecturersProf. Dr. **Florina M. Ciorba**

Prof. Dr. Heiko Schuldt

Prof. Dr. Christian Tschudin

Prof. Dr. Isabel Wagner

Assistants

Ali Ajorian

Thomas Jakobsche

Valentyna Pavliv

Yiming Wu

Info<https://adam.unibas.ch/go/crs/1995845>

Seminarraum 05.002

Spiegelgasse 5, Basel

Exercise 6: Introduction to MPI**(20 points)****Given:** November 13, 2025**Deadline:** November 23, 2025 (23:55)**Contact:** thomas.jakobsche@unibas.ch**Instructions**

- All exercise regulations discussed in the lecture and previous exercises apply.
- You will complete this exercise by executing programs on the miniHPC cluster.
- This exercise must be completed in groups of four and handed in by one.
- Submissions are made through ADAM and will be followed by interviews.
- Hand-in all files (e.g. scripts, codes, outputs) and a PDF with answers.

Objectives

- Hands-on work using an HPC system with a resource manager.
- Learn how to program, compile, and execute MPI code in C.
- Understand the usage of MPI to send messages between nodes.

Tasks

- Task 1: Multi-node "Hello World" in MPI (4 points)
- Task 2: Blocking MPI Send and Receive (4 points)
- Task 3: Non-blocking MPI Send and Receive (4 points)
- Task 4: Collective Communication with MPI Broadcast (4 points)
- Task 5: Synchronization with MPI Barrier (4 points)

How to use miniHPC

- You will work on miniHPC, a computing cluster for research and teaching purposes (see more <https://hpc.dmi.unibas.ch/research/miniHPC/>).
- Login: `ssh <shortname>@cl-login.dmi.unibas.ch` (UniBas short name and password, you need to be connected to the UniBas VPN). After logging in, you will be at your home directory on the miniHPC login node.
- The login node is the starting point to interact with the cluster, you should not execute your programs on the login node. Use the Slurm¹ resource manager to submit computing "jobs" to the multiple nodes of the cluster.

¹<https://slurm.schedmd.com>

- The Slurm resource manager coordinates your (and others) jobs and handles the allocation and scheduling of jobs onto nodes. Your jobs may wait in the job queue if nodes are busy with other jobs.
- To move files between your local machine and the miniHPC cluster, you can use the `scp` command and appropriate flags (e.g., `-r` to copy directories recursively).
- To run a program on compute nodes, request resources with a job script. Submit it using `sbatch <job_script>`, cancel jobs with `scancel <job_id>`, and check their status using `squeue`. Job output is saved in `slurm-<job_id>.out`.
- **It is OK to compile your programs on the login node, but do not execute them on the login node, always submit them as jobs.**

MPI instructions

- You can compile your MPI C program with MPICC (a wrapper around `gcc` that adds MPI headers and libraries) by first loading the newest installed MPICC compiler from the OpenMPI^a implementation with `module load OpenMPI/5.0.3-GCC-13.3.0` and then compiling with `mpicc -O3 my_program.c -o my_program`.
- *Hint:* What we may call processes, Slurm calls tasks, and MPI calls ranks.

^a<https://www.open-mpi.org>

Task 1: Multi-node "Hello World" in MPI

(4 points)

Write a C program that prints "Hello World from rank: <rank ID> on node: <hostname>" using multiple nodes. Write a job script to execute your program on miniHPC (see an example job script in Listing 1).

1. Write your C program using the appropriate MPI functions `MPI_Init`, `MPI_Finalize`, etc. Use `MPI_Get_processor_name` to get the hostname. (*Hint:* You can use the example from the lecture.)
2. Submit four jobs with different combinations of `nodes` $\in \{1, 2\}$ and `ntasks-per-node` $\in \{1, 2\}$. Record how many processes are launched and on which nodes.
3. Explain the resulting behavior. If our goal of using MPI is to exchange messages between nodes, which job configurations make sense and which don't?

Task 2: Blocking MPI Send and Receive

(4 points)

Write a C program to send a message between 4 MPI ranks on 4 different nodes. Message to be sent by one rank: "Hello World from rank: <rank ID> on node: <hostname>". Message to be printed by the other ranks: "Rank: <rank ID> on node: <hostname> received the following message: <message content>".

1. You can adjust your "Hello World" code of the previous task or write new code with 4 ranks in mind. Use `MPI_Send` on one rank and `MPI_Recv` on the others.
2. Adjust your job script to explicitly launch 4 MPI ranks on 4 different nodes (1 rank per node). Record the output of your program.
3. What happens if you increase `ntasks-per-node`? Explain the resulting behavior.

Task 3: Non-blocking MPI Send and Receive**(4 points)**

Use non-blocking point-to-point communication to send a message between 4 MPI ranks on 4 different nodes. Message to be sent by one rank: "Hello World from rank: <rank ID> on node: <hostname>". Message to be printed by the other ranks: "Rank: <rank ID> on node: <hostname> received the following message: <message content>".

1. You can adjust the code of the previous task or write new code. Use `MPI_Isend` on one rank and `MPI_Irecv` on the others. (*Hint*: Use `MPI_Wait` or `MPI_Test` to make sure everything works correctly).
2. Adjust your job script to explicitly launch 4 MPI ranks on 4 different nodes (1 rank per node). Record the output of your program.
3. What do you notice in the ordering of the output messages? Under which assumptions is non-blocking communication better than blocking communication?

Task 4: Collective Communication with MPI Broadcast**(4 points)**

Write a C program that uses MPI to distribute a message from one rank to all other ranks (using 4 MPI ranks on 4 different nodes). Message to be sent by one rank: "Hello World from rank: <rank ID> on node: <hostname>". Message to be printed by the other ranks: "Rank: <rank ID> on node: <hostname> received the following message: <message content>".

1. You can adjust the code of the previous task or write new code. Use `MPI_Bcast` to distribute the message to other ranks and then print.
2. How does this approach compare to blocking and non-blocking communication? What are the advantages and disadvantages?

Task 5: Synchronization with MPI Barrier**(4 points)**

Look at the C code in Listing 2, it contains an MPI program with multiple `MPI_Barriers`. You need to classify (and explain why) each barrier as:

- **Produces an error**, e.g., runtime error or deadlock
- **Implementation-wise OK**, but useless/illogical
- **Correct**, solves a real synchronization need

Listing 1: Example Job Script.

```
1  #!/bin/bash
2  #SBATCH --job-name=my_job      # The name of the job
3  #SBATCH --partition=xeon       # The main partition of miniHPC
4  #SBATCH --hint=nomultithread   # Disable hardware multithreading
5  #SBATCH --exclusive            # Exclusive access to resources
6  #SBATCH --time=00:10:00        # Requested execution time
7  #SBATCH --nodes=1              # Requested number of nodes
8  #SBATCH --ntasks-per-node=1    # Requested processes per node
9  #SBATCH --cpus-per-task=1      # Requested threads per process
10
11 # Tell Slurm to execute something (e.g. your code)
12 # We execute the hostname command as a placeholder
13 srun hostname
```

Listing 2: Example C code.

```
1  #include <mpi.h>
2  #include <stdio.h>
3  #include <unistd.h>
4
5  int main(int argc, char **argv) {
6
7      MPI_Barrier(MPI_COMM_WORLD); // TODO: check
8      MPI_Init(&argc, &argv);
9      MPI_Barrier(MPI_COMM_WORLD); // TODO: check
10
11     int rank, size, len;
12     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
13     MPI_Comm_size(MPI_COMM_WORLD, &size);
14
15     printf("Rank %d: Hello World\n", rank);
16     fflush(stdout);
17
18     usleep(150000 * (rank + 1));
19     if (rank == 0) { MPI_Barrier(MPI_COMM_WORLD); } // TODO: check
20     MPI_Barrier(MPI_COMM_WORLD); // TODO: check
21
22     char msg[128] = {0};
23     if (rank == 0) { snprintf(msg, sizeof(msg), "Root %d", rank); }
24     MPI_Bcast(msg, 128, MPI_CHAR, 0, MPI_COMM_WORLD);
25     MPI_Barrier(MPI_COMM_WORLD); // TODO: check
26
27     printf("Rank %d: got \"%s\"\n", rank, msg);
28     fflush(stdout);
29
30     MPI_Finalize();
31     MPI_Barrier(MPI_COMM_WORLD); // TODO: check
32
33     return 0;
34 }
```